



Corporación Universitaria del Huila - Corhuila

DEPENDENCY INVERSION PRINCIPLE

Presentado por:

Karina Cantillo Plaza

Laura Sofia Cangrejo

Carol Liceth Cruz Roa

Nicolas Garcia Quintero

Maria Fernanda Robayo

Daniela Zanabria

Presentado a:

Luis Angel Vargas Narvaez

Facultad de Ingeniería

VI Semestre

Arquitectura de Software

Neiva - Huila

2026

Tabla de contenido

1. Introducción	3
2. ¿Qué es el Dependency Inversion Principle (DIP)?	4
3. Aplicación del DIP en el Proyecto	5
4. Explicación de las Capas	6
4.1 Controller (Módulo de Alto Nivel)	6
4.2 Service (Interfaz - Abstracción)	6
4.3 ServiceImpl (Implementación - Bajo Nivel)	7
4.4 Repository	7
5. Beneficios Obtenidos al Aplicar DIP	8
6. Problema Presentado: Error 405 Method Not Allowed	9
7. Pruebas Realizadas en Postman	10
7.1 Prueba Método POST – Creación de Categoría	10
7.2 Prueba Método POST – Creación de Producto	11
7.3 Prueba Método GET – Listar Categorías	12
7.4 Prueba Método GET – Obtener por ID	13
7.5 Prueba Método GET – Listar Productos	14
7.6 Prueba Método PATCH – Actualización	15
7.7 Prueba Método DELETE – Eliminación	16
8. Conclusión	17

1. Introducción

En el desarrollo de software es fundamental diseñar sistemas que sean flexibles, mantenibles y escalables. Para lograr esto existen los principios SOLID, los cuales establecen buenas prácticas en la programación orientada a objetos y permiten construir aplicaciones mejor estructuradas.

En el presente trabajo, nuestro equipo aplicó uno de estos principios, el **Dependency Inversion Principle (DIP)** o Principio de Inversión de Dependencias, durante el desarrollo del proyecto *Task Movil API*. A través de la implementación de una arquitectura por capas (Controller, Service, ServiceImpl y Repository), buscamos reducir el acoplamiento entre los componentes del sistema y mejorar la organización del código.

Durante el desarrollo del proyecto, analizamos cómo estructurar correctamente las dependencias entre las clases, asegurándonos de que los módulos de alto nivel dependieran de abstracciones y no de implementaciones concretas. Esto nos permitió comprender de manera práctica cómo el principio DIP contribuye a la creación de aplicaciones más limpias, escalables y fáciles de mantener.

En este documento se explicará qué es el Dependency Inversion Principle, cómo fue aplicado en nuestro proyecto y cuáles fueron los beneficios obtenidos a partir de su implementación.

2. ¿Qué es el Dependency Inversion Principle (DIP)?

El **Dependency Inversion Principle (DIP)** o Principio de Inversión de Dependencias es uno de los cinco principios SOLID de la programación orientada a objetos. Su objetivo principal es reducir el acoplamiento entre las clases y mejorar la organización del sistema.

Este principio establece que los módulos de alto nivel no deben depender de módulos de bajo nivel, sino que ambos deben depender de abstracciones. Además, las abstracciones no deben depender de los detalles, sino que los detalles deben depender de las abstracciones.

En términos simples, significa que las clases principales del sistema no deben depender directamente de clases concretas, sino de **interfaces**, permitiendo que las implementaciones puedan cambiar sin afectar otras partes del sistema.

En el contexto de aplicaciones desarrolladas con **Spring Boot**, este principio se aplica mediante el uso de interfaces y la **inyección de dependencias**, lo que permite desacoplar las diferentes capas del sistema y mantener una arquitectura más organizada y flexible.

3. Aplicación del DIP en el Proyecto

En el proyecto *Task Movil API* se aplicó el principio DIP organizando la estructura en diferentes capas:

- Controller (Capa de alto nivel)
- Service (Interfaz - Abstracción)
- ServiceImpl (Implementación)
- Repository (Acceso a datos)

La relación entre estas capas funciona de la siguiente manera:

1. El Controller recibe la petición HTTP.
2. El Controller llama al Service.
3. El Service define el contrato mediante una interfaz.
4. La implementación del Service ejecuta la lógica.
5. El Repository se comunica con la base de datos.

Lo importante es que el Controller no depende directamente de la clase concreta del Service, sino de la interfaz.

Ejemplo en el Controller:

```
private final CategoriaService service;
```

Aquí el Controller depende de la abstracción *CategoriaService*, no de la implementación concreta.

4. Explicación de las Capas

4.1 Controller (Módulo de Alto Nivel)

El Controller es la capa que maneja las peticiones HTTP como:

- GET
- POST
- PUT
- DELETE

Ejemplo:

@PostMapping

```
public CategoriaResponseDto create(@Valid @RequestBody CategoriaCreateDto dto)
{
    return service.create(dto);
}
```

El Controller no contiene lógica de negocio.

Solo recibe la petición y delega la responsabilidad al Service.

Es considerado un módulo de alto nivel porque coordina el flujo principal del sistema.

4.2 Service (Interfaz - Abstracción)

El Service es una interfaz que define las operaciones disponibles, por ejemplo:

```
public interface CategoriaService {
    CategoriaResponseDto create(CategoriaCreateDto dto);
    List<CategoriaResponseDto> findAll();
}
```

Aquí se define qué se puede hacer, pero no cómo se hace.

Esto representa la abstracción en el principio DIP.

4.3 ServiceImpl (Implementación - Bajo Nivel)

La clase ServiceImpl implementa la interfaz:

@Service

```
public class CategoriaServiceImpl implements CategoriaService {
```

En esta clase se encuentra:

- La lógica de negocio
- La comunicación con el Repository
- El acceso a la base de datos

Es considerada un módulo de bajo nivel porque contiene los detalles de implementación.

4.4 Repository

El Repository se encarga de interactuar directamente con la base de datos.

Aquí se realizan operaciones como guardar, actualizar, eliminar o consultar registros.

5. Beneficios Obtenidos al Aplicar DIP

Al aplicar el Principio de Inversión de Dependencias en el proyecto se lograron los siguientes beneficios:

- Menor acoplamiento entre las capas.
- Mayor claridad en la estructura del proyecto.
- Código más organizado.
- Mayor facilidad de mantenimiento.
- Facilidad para realizar pruebas unitarias.
- Posibilidad de cambiar la implementación sin afectar el Controller.

Por ejemplo, si en el futuro se cambia la forma en que se guardan los datos, el Controller no necesita modificarse.

6. Problema Presentado: Error 405 Method Not Allowed

Durante las pruebas del endpoint de actualización se presentó el siguiente error:

```
{
  "timestamp": "2026-02-11T15:25:41.947+00:00",
  "status": 405,
  "error": "Method Not Allowed",
  "path": "/api/categorias/1"
}
```

El error 405 significa que la URL existe, pero el método HTTP utilizado no está permitido para ese endpoint.

En el Controller se tenía configurado:

```
@PutMapping("/{id}")
```

Sin embargo, la petición se estaba realizando con el método PATCH, lo que generaba el error.

Al cambiar la petición para que coincidiera con el método correcto (PATCH o PUT según la configuración), el problema se solucionó.

Este error no está relacionado con el DIP, sino con la configuración del método HTTP.

7. Pruebas Realizadas en Postman

Para validar el correcto funcionamiento de la API desarrollada en Spring Boot, se realizaron pruebas utilizando Postman.

Se probaron los endpoints correspondientes a las entidades **Categorías** y **Productos**, verificando los métodos HTTP POST, GET, PATCH y DELETE. Todas las pruebas retornaron código de estado **200 OK**, lo que indica que las operaciones fueron ejecutadas correctamente.

7.1 Prueba Método POST

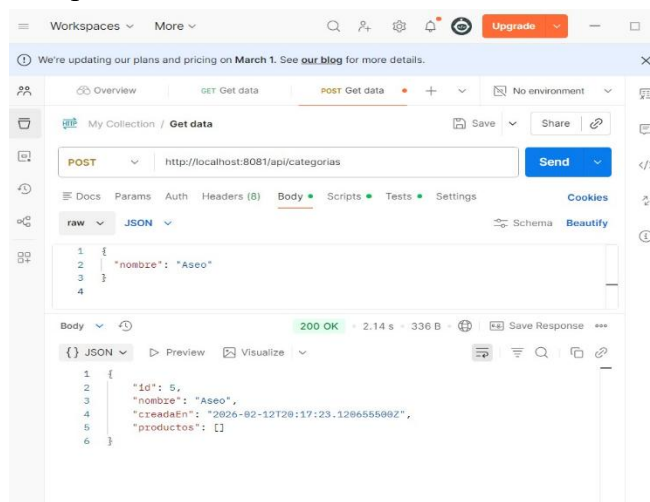
Creación de Categoría

- URL: `http://localhost:8081/api/categorias`
- Método: POST
- Body (JSON):

```
{  
  "nombre": "Aseo"  
}
```

Resultado:

- Se creó correctamente la categoría.
- El sistema generó automáticamente el id y la fecha creadaEn.
- Código de respuesta: **200 OK**



7.2 Prueba Método POST

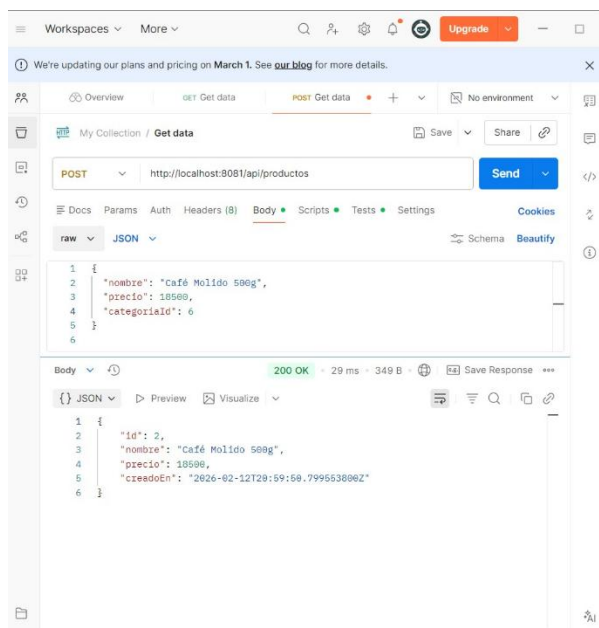
Creación de Producto

- URL: `http://localhost:8081/api/productos`
- Método: POST
- Body (JSON):

```
{  
  "nombre": "Café Molido 500g",  
  "precio": 18500,  
  "categoriaId": 6  
}
```

Resultado:

- El producto fue creado correctamente.
- Se asignó automáticamente el id y la fecha creadoEn.
- Código de respuesta: **200 OK**



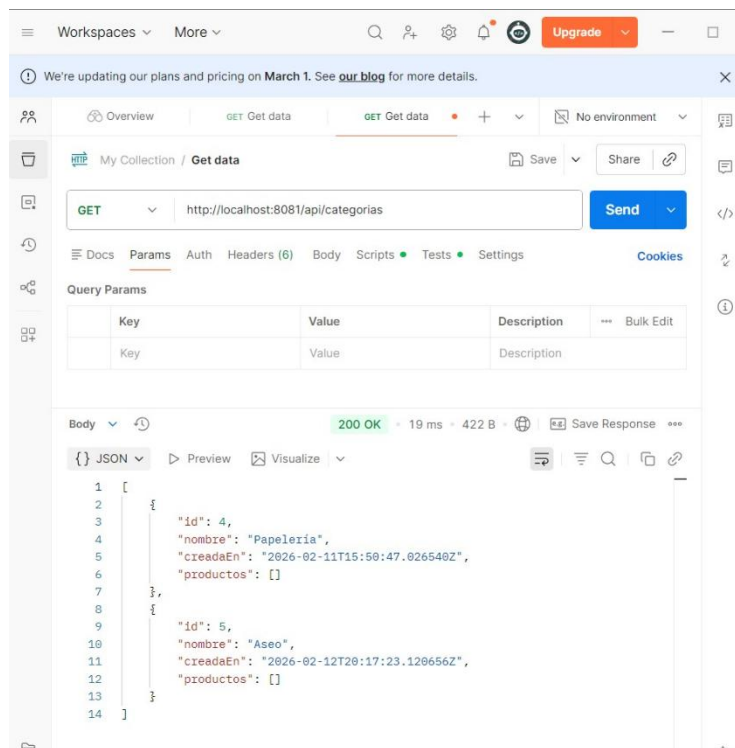
7.3 Prueba Método GET

Obtener todas las Categorías

- URL: `http://localhost:8081/api/categorias`
- Método: GET

Resultado:

- Se obtuvo una lista de categorías registradas.
- Se visualizaron los campos `id`, `nombre`, `creadaEn` y la relación `productos`.
- Código de respuesta: **200 OK**



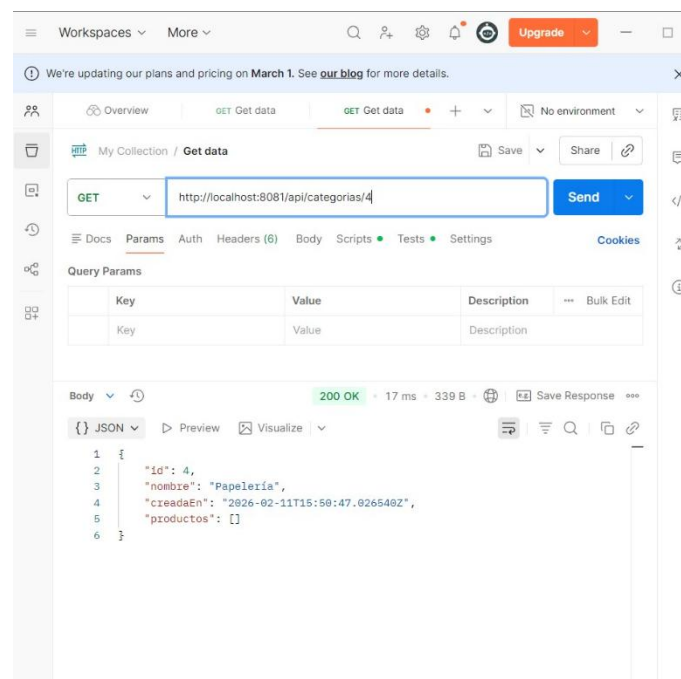
7.4 Prueba Método GET

Obtener Categoría por ID

- URL: `http://localhost:8081/api/categorias/4`
- Método: GET

Resultado:

- Se obtuvo correctamente la categoría con id 4.
- Código de respuesta: **200 OK**



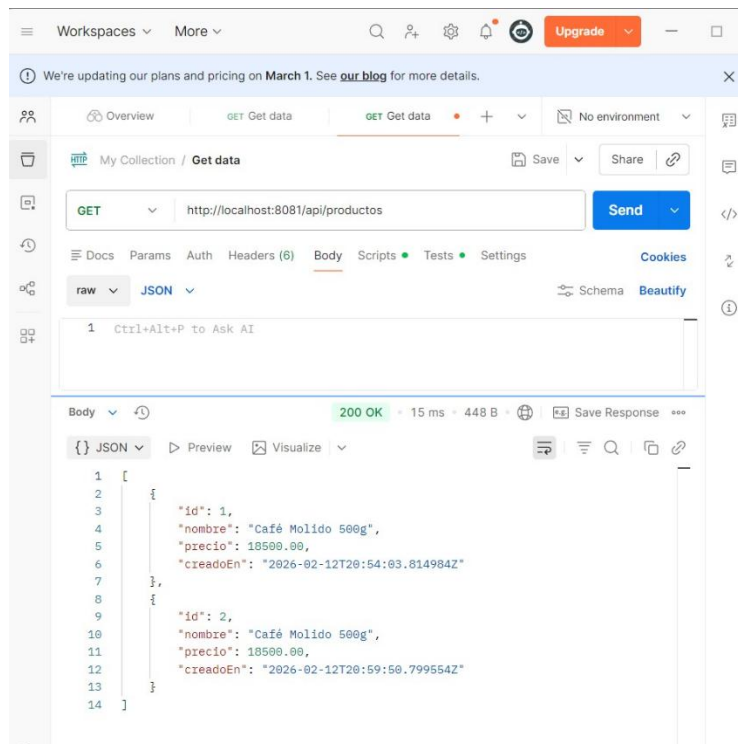
7.5 Prueba Método GET

Obtener todos los Productos

- URL: `http://localhost:8081/api/productos`
- Método: GET

Resultado:

- Se listaron los productos almacenados en la base de datos.
- Código de respuesta: **200 OK**



7.6 Prueba Método PATCH

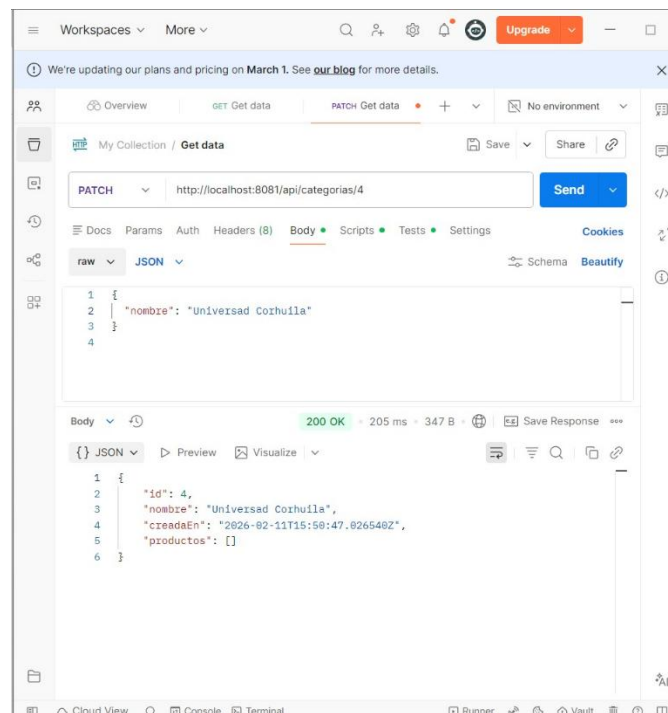
(Actualización)

- URL: `http://localhost:8081/api/categorias/4`
- Método: PATCH
- Body (JSON):

```
{  
  "nombre": "Universidad Corhuila"  
}
```

Resultado:

- La categoría fue actualizada correctamente.
- El nombre cambió sin afectar otros campos.
- Código de respuesta: **200 OK**

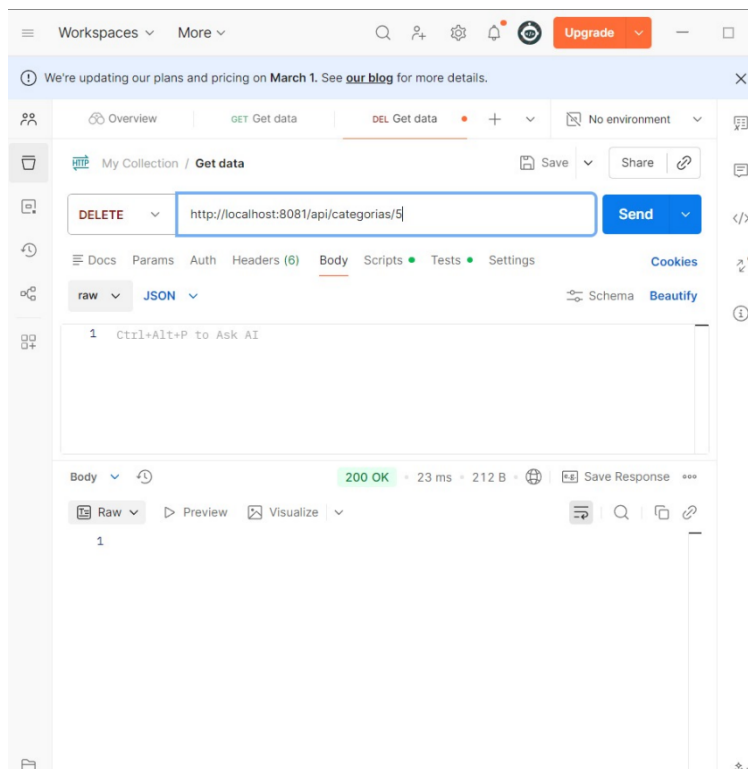


7.7 Prueba Método DELETE

- URL: `http://localhost:8081/api/categorias/5`
- Método: DELETE

Resultado:

- La categoría fue eliminada correctamente.
- Código de respuesta: **200 OK**



7 Conclusión

El Dependency Inversion Principle es un principio fundamental dentro de SOLID que promueve el desacoplamiento entre las capas del sistema.

En el proyecto Task Movil API se aplicó correctamente mediante el uso de interfaces en la capa Service, permitiendo que el Controller dependa de una abstracción y no de una implementación concreta.

Gracias a este principio, el proyecto tiene una estructura más organizada, flexible y fácil de mantener, lo cual es esencial en el desarrollo de aplicaciones modernas.